Capstone Details

SRE Case Studies

Case 1

E-Commerce Platform Reliability Enhancement

Background Story

A leading e-commerce platform, **ShopMax**, was experiencing **frequent outages and slow performance** during peak sales events. Customers complained about slow page loads, failed transactions, and unexpected service downtimes. The engineering team struggled with **high operational toil**, spending hours manually resolving incidents.

The company's **Black Friday sale** was around the corner, and the leadership team was concerned about **losing millions in revenue** due to system failures. The **SRE team was called in** to diagnose the issues, propose solutions, and ensure a **highly available, scalable, and resilient infrastructure**.

Through initial analysis, the SRE team discovered:

1. **Service failures under peak traffic loads.**
2. **Inconsistent database performance, causing API slowdowns.**
3. **Auto-scaling delays leading to service disruptions.**
4. **Ineffective monitoring, leading to delayed incident responses.**

The company's executive team **challenged the SREs** to design and implement a solution before the Black Friday sale, ensuring a **seamless shopping experience** for millions of users.

Challenge Statement:

1. How can we **set up accurate SLI/SLO metrics and dashboards** to track system health and performance?
2. How can we **automate infrastructure provisioning and deployments** to improve scalability and resilience?
3. How can we **use chaos engineering** to test system failure scenarios and identify weaknesses?
4. How can we **analyze past incidents** to improve future incident response and mitigation strategies?

🚀 Participants are expected to work on the deliverables:

- Define **SLI/SLO metrics** and create monitoring dashboards.
- Automate cloud infrastructure provisioning and deployments.
- Run **chaos experiments** and document resilience gaps.
- Conduct **root cause analysis (RCA)** and propose improvements.

For E-Commerce Platform Reliability Enhancement

♢ Amazon Prime Day Outage (2018)

- **Issue:** Amazon's backend systems couldn't handle the traffic surge, leading to **cart failures, slow page loads, and payment processing issues**.

Case -2

Financial Services - High Availability & Resilience

Background Story

A global financial institution, **FinBank**, provides **real-time trading services** to customers worldwide. Due to the nature of **high-frequency trading**, even a **few milliseconds of latency** can result in **millions of dollars in losses**. Recently, the company has experienced **multiple outages**, leading to **customer dissatisfaction and regulatory concerns**.

During a recent **market volatility event**, FinBank's trading application **slowed down significantly**, causing transactions to fail or execute at incorrect prices. This resulted in **trader losses and reputational damage**. The **IT leadership escalated the issue to the SRE team**, asking for a **comprehensive analysis and immediate solutions**.

Preliminary findings showed:

1. **Database queries were slowing down under peak load**, leading to transaction failures.
2. **Service failover mechanisms were ineffective**, increasing downtime.
3. **Network latency spikes** were affecting real-time price updates.
4. **Existing monitoring lacked predictive capabilities**, leading to reactive incident handling.

Challenge Statement:

1. How can we **define and track real-time SLI/SLO metrics** to improve trading performance?
2. How can we **automate database scaling and failover** to minimize downtime?
3. How can we **simulate real-world failures using chaos engineering** and analyze system weaknesses?
4. How can we **improve incident response workflows** to ensure faster resolution times?

🚀 Participants are expected to work on the deliverables:

- Define **SLI/SLO metrics** and configure real-time monitoring dashboards.

- Automate infrastructure scaling and failover mechanisms.
- Design and execute **chaos experiments** to test system resilience.
- Conduct **root cause analysis (RCA)** and propose future improvements.

For Financial Services - High Availability & Resilience

◈ **Robinhood Trading Outage (2020)**

- **Issue:** During a major market movement, Robinhood experienced a **multi-day outage**, preventing users from trading.

Case 3:

Uber - Scaling for High Availability & Latency Optimization

Background Story

Uber operates in over 10,000 cities worldwide, facilitating **millions of ride requests per day**. With such an enormous scale, **latency, availability, and real-time processing** become mission critical. A fraction of a second delay in the driver matching system or payment processing can lead to **revenue loss, customer dissatisfaction, and operational inefficiencies**.

During peak times, such as **New Year's Eve or a city-wide event**, the ride-matching service experiences a **5-10x traffic surge**, overwhelming backend systems. Despite Uber's cloud-native architecture, challenges arise:

1. **Inconsistent ride request handling:** Some users experience long wait times, while others see repeated ride cancellations.
2. **Database bottlenecks:** The fare estimation and trip calculation services slow down due to high concurrent queries.

3. **Network latency issues:** Real-time location updates lag, impacting driver-passenger coordination.
4. **Slow auto-scaling:** The current scaling strategy is reactive, leading to service degradation before resources catch up.

In a **real-world scenario**, Uber faced a **global service outage in 2019** due to **database replication failures**, causing app downtime across multiple regions. A lack of proactive failover strategies meant **riders couldn't book trips**, leading to a massive revenue loss in just a few hours.

The **SRE team** at Uber must design a **highly available, fault-tolerant, and scalable system** to ensure that ride-hailing remains **seamless**, even under extreme traffic spikes.

Challenge Statement:

1. How can we **define real-time SLI/SLO metrics** for ride-matching, driver availability, and API response times?
2. How can we **optimize auto-scaling** for unpredictable demand surges?
3. How can we **simulate failures (e.g., regional server crashes, API downtime)** using chaos engineering?
4. How can we **analyze and improve incident response** for faster ride recovery?

🚀 Participants are expected to work on the deliverables:

- Define **SLI/SLO metrics** for real-time API performance.
- Automate **scaling mechanisms** and **infrastructure provisioning**.
- Conduct **chaos engineering experiments** to test service degradation.
- Perform **RCA on a simulated incident** and propose improvements.

Case 4

Swiggy - Reliability in Food Delivery Operations

Background Story

Swiggy operates in **hundreds of cities**, processing **millions of food orders daily**. Customers expect food delivery in under **30 minutes**, which requires **precise coordination between restaurants, delivery partners, and customers**. However, operational disruptions can severely impact Swiggy's service reliability and brand trust.

Consider a **Friday night dinner rush** where **order volumes spike 5x**. During such events, Swiggy faces **several reliability challenges**:

1. **Delivery tracking API failures:** Users see outdated tracking information or blank maps due to server overload.
2. **Order assignment failures:** The system fails to efficiently match drivers with orders due to backend congestion.
3. **Inconsistent restaurant availability:** Some restaurants appear offline due to cache synchronization delays.
4. **Delayed push notifications:** Customers don't receive updates, leading to confusion and high cancellation rates.

A real-world example occurred in **2021**, when Swiggy experienced **widespread order failures** due to a **database deadlock issue**, leading to **thousands of incomplete or delayed orders**. The incident impacted **customer trust**, forcing the company to roll out **compensations and refunds**.

The **SRE team at Swiggy** must design a system that can **dynamically scale, predict failures, and provide real-time insights** into performance metrics to ensure a **seamless food delivery experience**.

Challenge Statement:

1. How can we **define and monitor critical SLIs/SLOs** (e.g., order fulfillment time, API response, app uptime)?
2. How can we **use automation to optimize order assignment** and prevent failures?
3. How can we **simulate failures (e.g., slow database queries, driver location mismatches)** using chaos engineering?
4. How can we **improve incident response workflows** to ensure minimal disruptions?

🚀 Participants are expected to work on the deliverables:

- Define **SLI/SLO metrics** for key delivery operations.
- Implement **auto-scaling and infrastructure automation**.
- Run **chaos engineering tests** to validate system resilience.
- Conduct **incident analysis and propose reliability improvements.**